

Chapter 4.

FINE-TUNE MODELS AND TRAINING ALGORITHMS

PhD. Nguyễn Thị Khánh Tiên
tienntk@ut.edu.vn



Finetune model

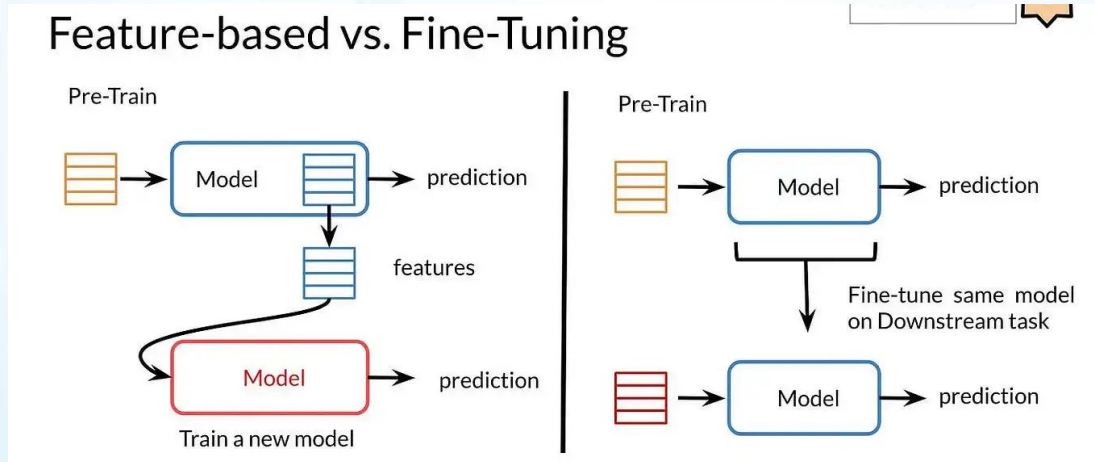
Finetuning is a process in machine learning where you take a model that has already been trained on a large dataset (the pre-trained model) and further train it on a smaller, task-specific dataset.

The goal is to adapt the pre-trained model's learned features to a new, related task, leading to better performance with less data and faster training times compared to training a model from scratch.

Finetuning is a key technique in **transfer learning**.

Why Finetune?

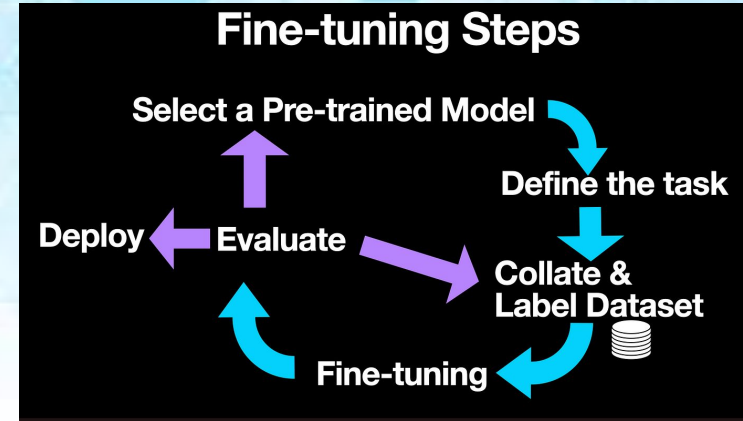
- **Leveraging Learned Features:** Pre-trained models have learned general features from massive datasets. Finetuning allows you to utilize these features, which are often relevant to your specific task.
- **Reduced Data Requirements:** You typically need significantly less data to finetune a model than to train one from scratch.
- **Faster Training:** Since the model is already partially trained, the finetuning process usually converges faster.
- **Improved Performance:** In many cases, finetuned models achieve higher accuracy and better generalization on the target task.



Finetune model

Steps Involved in Finetuning:

1. **Choose a Pre-trained Model:** Select a model that has been trained on a large dataset and whose architecture is suitable for the target task.
2. **Prepare Your Dataset:** Gather and preprocess the data specific to your task.
3. **Modify the Model:** Adapt the pre-trained model's architecture if necessary, typically by replacing the final classification layer.
4. **Freeze Layers (Optional but Common):** Initially freeze the weights of the early layers of the pre-trained model to prevent them from being drastically changed by the new, smaller dataset.
5. **Train the Model:** Train the modified model on your dataset, typically using a lower learning rate than you would for training from scratch.
6. **Unfreeze and Retrain (Optional):** After initial training, you might unfreeze some of the earlier layers and continue training with an even lower learning rate to fine-tune the entire model.
7. **Evaluate Performance:** Assess the performance of the finetuned model on a validation set.
8. **Hyperparameter Tuning:** Adjust hyperparameters like learning rate, batch size, and the number of frozen layers to optimize performance.



Finetune model

Key Considerations for Finetuning:

- **Similarity of Datasets:** The more similar your target task and dataset are to the original task and dataset the pre-trained model was trained on, the better the finetuning will likely work.
- **Size of the New Dataset:** The size of your task-specific dataset will influence how many layers you should unfreeze and the learning rate you should use. Smaller datasets might benefit from freezing more layers to prevent overfitting.
- **Computational Resources:** Finetuning can still be computationally intensive, especially for large pre-trained models.
- **Potential for Catastrophic Forgetting:** If you finetune too aggressively on a very different task, the model might "forget" the useful general features it learned during pre-training.

When to Finetune:

- You have a limited amount of labeled data for your specific task.
- A good pre-trained model exists for a related task or domain.
- You want to achieve good performance quickly without training a large model from scratch.

Fine-Tuning Options Comparison

Local Fine-Tuning

Advantages

- Full control
- Data privacy
- No recurring fees

Disadvantages

- High initial cost
- Resource limits
- Expertise required

Rented GPU Options

Advantages

- Scalable resources
- Lower upfront cost

Disadvantages

- Ongoing expenses
- Potential time limits

Fine-Tuning Services

Advantages

- User-friendly
- Managed infrastructure

Disadvantages

- Limited control
- Data security concerns
- Possible higher costs

Training Algorithms

Training algorithms are the methods used to teach a machine learning model to learn from data. They define how the model's internal parameters (weights and biases in neural networks) are adjusted based on the training data to minimize a defined **loss function**. The loss function measures the difference between the model's predictions and the actual target values.

Core Components of a Training Algorithm:

1. **Loss Function (Objective Function):** A function that quantifies the error or discrepancy between the model's predictions and the true values in the training data. The goal of training is to minimize this function. Examples include:
 - **Mean Squared Error (MSE):** For regression tasks.
 - **Binary Cross-Entropy:** For binary classification.
 - **Categorical Cross-Entropy:** For multi-class classification.
2. **Optimizer:** An algorithm that determines how the model's parameters are updated to reduce the loss function. Common optimizers include:
 - **Gradient Descent (GD):** A basic optimization algorithm that iteratively moves the parameters in the direction of the negative gradient of the loss function.
 - **Stochastic Gradient Descent (SGD):** Updates parameters using the gradient calculated on a single randomly chosen training example (or a small batch). This is more efficient for large datasets.
 - **Adam (Adaptive Moment Estimation):** An adaptive learning rate optimization algorithm that is widely used and often performs well.
 - **RMSprop (Root Mean Square Propagation):** Another adaptive learning rate optimizer.
 - **Adagrad (Adaptive Gradient Algorithm):** Adapts the learning rate for each parameter based on the historical gradients.
3. **Learning Rate:** A hyperparameter that controls the step size at each iteration while moving towards a minimum of the loss function. A high learning rate might lead to overshooting the minimum, while a low learning rate might result in slow convergence.
4. **Batch Size:** The number of training examples used in one iteration to calculate the gradient and update the model's parameters.
5. **Number of Epochs:** The number of times the entire training dataset is passed through the model during training.

Training Algorithms

Common Training Algorithms (Optimization Algorithms):

- **Gradient Descent and its Variants (SGD, Mini-batch GD):** These form the foundation of many training algorithms, especially for neural networks. They iteratively adjust model parameters based on the gradient of the loss function.
- **Backpropagation:** An algorithm used to efficiently calculate the gradients of the loss function with respect to the weights in a neural network. It's a crucial part of training deep learning models.
- **Evolutionary Algorithms (e.g., Genetic Algorithms):** While less common for training deep learning models directly, they can be used for tasks like hyperparameter optimization or neural architecture search.

Advanced Training Techniques:

- **Learning Rate Scheduling:** Adjusting the learning rate during training (e.g., decreasing it over time) can help the model converge better.
- **Regularization (L1, L2, Dropout):** Techniques to prevent overfitting by adding a penalty to the loss function or randomly dropping out neurons during training.
- **Batch Normalization:** A technique to stabilize and accelerate training by normalizing the activations of intermediate layers.
- **Early Stopping:** Monitoring the performance on a validation set and stopping training when the performance starts to degrade to prevent overfitting.
- **Data Augmentation:** Creating artificial variations of the training data to increase its size and improve the model's generalization ability.

Relationship Between Finetuning and Training Algorithms.

Finetuning is a specific application of the general training process. The key differences in finetuning often lie in:

- **Initialization:** The model's weights are initialized with the values learned during pre-training, rather than random initialization.
- **Layer Freezing/Unfreezing:** You strategically choose which layers to update during training.
- **Learning Rate Adjustment:** You often use different learning rates for different parts of the model.

Optimization algorithm in Pytorch

In PyTorch, training a neural network typically involves using an **optimization algorithm** to update the model's parameters (weights and biases) based on the gradients of a loss function with respect to those parameters. These algorithms aim to minimize the loss function, thereby improving the model's performance on the training data. PyTorch provides a rich set of optimization algorithms within its **torch.optim** module.

The choice of optimization algorithm can significantly impact the training process and the final performance of your model. There's no single "best" optimizer for all tasks. Here are some general guidelines:

- **AdamW** is often a good starting point for many modern deep learning tasks and architectures.
- **Adam** is also a very popular and generally effective choice.
- **SGD** with momentum can work well, especially with careful tuning of the learning rate and other hyperparameters. It might generalize better in some cases but often takes longer to converge.
- **RMSprop** is another good alternative to Adam.
- **Adagrad** and **Adadelta** were more popular in the past but are less commonly used now compared to Adam and its variants.
- **LBFGS** is often used for problems where you can afford full-batch training and need faster convergence in terms of the number of iterations (e.g., certain types of optimization problems in computer vision or physics)

1. Stochastic Gradient Descent (SGD):

- **Concept:** The most basic and fundamental optimization algorithm. It updates the model's parameters in the direction of the negative gradient of the loss function computed on a single random sample (or a small batch) of the training data.
- **Pros:** Simple to understand and implement.
- **Cons:** Can be slow to converge, especially with noisy gradients. May get stuck in local minima. The learning rate is crucial and often needs careful tuning.
- **PyTorch Implementation:** `torch.optim.SGD(params, lr=0.01, momentum=0, dampening=0, weight_decay=0, nesterov=False)`
 - **lr:** Learning rate.
 - **momentum:** Helps accelerate SGD in the relevant direction and dampens oscillations.
 - **weight_decay:** L2 regularization to prevent overfitting.
 - **nesterov:** Enables Nesterov momentum, which often leads to faster convergence.

Optimization algorithm in Pytorch

2. Adam (Adaptive Moment Estimation):

- **Concept:** An adaptive learning rate optimization algorithm that combines the benefits of both AdaGrad and RMSprop. It maintains per-parameter learning rates that are adapted based on estimates of the first and second moments of the gradients.
- **Pros:** Generally converges faster than SGD and requires less hyperparameter tuning. Effective for a wide range of problems.
- **Cons:** Can sometimes generalize worse than SGD in certain scenarios.
- **PyTorch Implementation:** `torch.optim.Adam(params, lr=0.001, betas=(0.9, 0.999), eps=1e-08, weight_decay=0, amsgrad=False)`
 - **lr:** Learning rate.
 - **betas:** Coefficients used for computing running averages of the gradient and its square.
 - **eps:** Term added to improve numerical stability.
 - **amsgrad:** Whether to use the AMSGrad variant of this algorithm.

3. RMSprop (Root Mean Square Propagation):

- **Concept:** Another adaptive learning rate algorithm that maintains a moving average of the squared gradients for each parameter. It divides the learning rate for each parameter by the square root of this average, effectively reducing the learning rate for parameters with large gradients.
- **Pros:** Often performs well in practice and is less prone to getting stuck in saddle points compared to SGD.
- **Cons:** Can sometimes converge slower than Adam.
- **PyTorch Implementation:** `torch.optim.RMSprop(params, lr=0.01, alpha=0.99, eps=1e-08, weight_decay=0, momentum=0, centered=False)`
 - **lr:** Learning rate.
 - **alpha:** Smoothing constant.
 - **centered:** If True, compute the centered RMSProp, the gradient is normalized by an estimation of its variance.

Optimization algorithm in Pytorch

4. Adagrad (Adaptive Gradient Algorithm):

- **Concept:** An adaptive learning rate algorithm that adapts the learning rate to the parameters, giving higher learning rates to infrequently updated parameters and lower learning rates to frequently updated parameters. It accumulates the squared gradients for each parameter over time.
- **Pros:** Suitable for sparse data.
- **Cons:** The learning rate can become very small over time, leading to slow convergence or even stopping prematurely.
- **PyTorch Implementation:** `torch.optim.Adagrad(params, lr=0.01, lr_decay=0, weight_decay=0, initial_accumulator_value=0, eps=1e-10)`
 - `lr`: Learning rate.
 - `lr_decay`: Learning rate decay per step.

5. Adadelata (Adaptive Delta):

- **Concept:** An extension of Adagrad that addresses its diminishing learning rate problem. Instead of accumulating all past squared gradients, it restricts the window of accumulated past gradients to a fixed size. It also doesn't require manual tuning of a global learning rate.
- **Pros:** Often performs well without needing to tune the learning rate.
- **Cons:** Can sometimes oscillate and may not converge as quickly as other adaptive methods in some cases.
- **PyTorch Implementation:** `torch.optim.Adadelata(params, lr=1.0, rho=0.9, eps=1e-06, weight_decay=0)`
 - `rho`: Coefficient used for computing a running average of squared gradients.

Optimization algorithm in Pytorch

6. AdamW (Adam with Weight Decay):

- **Concept:** A modification of the Adam optimizer that decouples the weight decay (L2 regularization) from the gradient update. This has been shown to often lead to better generalization performance compared to the standard Adam optimizer where weight decay is applied directly to the gradients.
- **Pros:** Often outperforms Adam in terms of generalization. Highly recommended for many modern deep learning architectures.
- **Cons:** None significant compared to Adam.
- **PyTorch Implementation:** `torch.optim.AdamW(params, lr=0.001, betas=(0.9, 0.999), eps=1e-08, weight_decay=0.01, amsgrad=False)`
 - Parameters are similar to Adam, with `weight_decay` being a separate parameter.

7. LBFGS (Limited-memory BFGS):

- **Concept:** A quasi-Newton method that approximates the Hessian matrix to guide the optimization process. It's a more sophisticated optimization algorithm that can often converge in fewer iterations than gradient-based methods, especially for smaller datasets and well-conditioned problems.
- **Pros:** Can converge quickly for certain types of problems.
- **Cons:** Requires computing and storing Hessian approximations, which can be memory-intensive for very large models. Typically used for full-batch optimization and might not be suitable for large datasets or mini-batch training.
- **PyTorch Implementation:** `torch.optim.LBFGS(params, lr=1, max_iter=20, max_eval=None, tolerance_grad=1e-05, tolerance_change=1e-09, history_size=100, line_search_fn=None)`
 - Has different parameters as it's a second-order optimization method.

Learning Rate Scheduling in Pytorch

Learning rate scheduling in PyTorch is a technique to adjust the learning rate of your optimizer during training. Instead of using a constant learning rate throughout the entire training process, you can dynamically change it based on the number of epochs, the performance on a validation set, or other criteria.

Why Use Learning Rate Scheduling?

- **Improved Convergence:** Starting with a higher learning rate can help the model quickly move towards a good region in the weight space. Then, reducing the learning rate allows for finer adjustments and helps the model converge to a better minimum.
- **Avoiding Local Minima:** A fluctuating or decreasing learning rate can help the model escape shallow local minima.
- **Better Generalization:** Carefully scheduled learning rates can sometimes lead to models that generalize better to unseen data.
- **Faster Training:** By starting with a higher learning rate, you might reach a reasonable performance level faster.

PyTorch `torch.optim.lr_scheduler` Module

PyTorch provides a dedicated module, `torch.optim.lr_scheduler`, which implements several common learning rate scheduling strategies.

```
from torch.optim import lr_scheduler
```

Common Learning Rate Schedulers in PyTorch

Here are some of the most commonly used learning rate schedulers in PyTorch:

1. **StepLR:** Reduces the learning rate by a fixed factor at specified epochs

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
scheduler = lr_scheduler.StepLR(optimizer, step_size=30,
gamma=0.1)
```

`step_size`: Number of epochs after which learning rate will be reduced.

`gamma`: Multiplicative factor of learning rate decay.

Learning Rate Scheduling in Pytorch

Common Learning Rate Schedulers in PyTorch:

1. **StepLR**: Reduces the learning rate by a fixed factor at specified epochs

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)  
scheduler = lr_scheduler.StepLR(optimizer, step_size=30, gamma=0.1)  
# step_size: Number of epochs after which learning rate will be reduced.  
# gamma: Multiplicative factor of learning rate decay.
```

2. **MultiStepLR**: Reduces the learning rate by a fixed factor at specified epoch milestones.

```
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)  
milestones = [50, 100, 150]  
scheduler = lr_scheduler.MultiStepLR(optimizer, milestones=milestones, gamma=0.1)  
# milestones: List of epoch indices. Must be increasing.  
# gamma: Multiplicative factor of learning rate decay.
```

3. **ExponentialLR**: Reduces the learning rate by an exponential factor.

```
optimizer = torch.optim.RMSprop(model.parameters(), lr=0.01)  
scheduler = lr_scheduler.ExponentialLR(optimizer, gamma=0.9)  
# gamma: Multiplicative factor of learning rate decay (should be < 1).
```

4. **CosineAnnealingLR**: Reduces the learning rate following a cosine annealing schedule

```
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)  
T_max = 100 # Maximum number of iterations in one cycle.  
scheduler = lr_scheduler.CosineAnnealingLR(optimizer, T_max=T_max)  
# T_max: Maximum number of iterations in one cycle.  
# eta_min: Minimum learning rate during the cycle (default: 0).
```

5. ReduceLROnPlateau: Reduces the learning rate when a metric has stopped improving. This scheduler monitors a metric (usually validation loss or accuracy)

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
scheduler = lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=10)
# mode: 'min' (for loss) or 'max' (for accuracy).
# factor: Factor by which the learning rate will be reduced.
# patience: Number of epochs with no improvement after which learning rate will be reduced.
```

6. CyclicLR: Cyclically varies the learning rate between two boundaries.

```
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
base_lr = 0.001
max_lr = 0.1
step_size_up = 5 # Number of training iterations in the increasing half of a cycle.
scheduler = lr_scheduler.CyclicLR(optimizer, base_lr=base_lr, max_lr=max_lr, step_size_up=step_size_up, mode='triangular')
# base_lr: Initial learning rate which is the lower boundary in the cycle.
# max_lr: Upper boundary in the cycle.
# step_size_up: Number of training iterations in the increasing half of a cycle.
# mode: {'triangular', 'triangular2', 'exp_range'}.
```

7. OneCycleLR: Adjusts the learning rate following a 1-cycle policy. This policy involves increasing the learning rate from a low value to a maximum value and then decreasing it again

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
total_steps = len(dataloader) * num_epochs
scheduler = lr_scheduler.OneCycleLR(optimizer, max_lr=0.1, total_steps=total_steps)
# max_lr: Upper learning rate boundaries in the cycle.
# total_steps: The total number of steps in the training loop across all epochs.
# epochs: The number of epochs to train for.
# steps_per_epoch: The number of steps per epoch (length of the dataloader).
```

Learning Rate Scheduling in Pytorch

How to Use a Learning Rate Scheduler

Here's the general workflow for using a learning rate scheduler in your training loop:

1. **Initialize the Optimizer:** Create your optimizer as usual.
2. **Initialize the Scheduler:** Instantiate the desired learning rate scheduler, passing the optimizer as an argument, along with any specific parameters for that scheduler.
3. **In the Training Loop:**
 - Perform a training step (forward pass, loss calculation, backward pass, `optimizer.step()`).
 -
 - Crucially, call `scheduler.step()` after each optimization step (for schedulers like **StepLR**, **MultiStepLR**, **ExponentialLR**, **CosineAnnealingLR**, **CyclicLR**, **OneCycleLR**) or after each epoch (for **ReduceLROnPlateau**).

Important Considerations:

- **When to call `scheduler.step()`:**
 - For schedulers like **StepLR**, **MultiStepLR**, **ExponentialLR**, **CosineAnnealingLR**, **CyclicLR**, and **OneCycleLR**, you typically call `scheduler.step()` **after** each `optimizer.step()`.
 - For **ReduceLROnPlateau**, you should call `scheduler.step(metric)` after each epoch, passing the value of the metric you are monitoring (e.g., validation loss).
- **Choosing the Right Scheduler:** The best scheduler depends on your specific problem, model architecture, and dataset. Experimentation is often necessary.
- **Hyperparameter Tuning:** The parameters of the learning rate scheduler (e.g., `step_size`, `gamma`, `patience`) are also hyperparameters that might need tuning.
-
- **Monitoring Learning Rate:** It's often helpful to log the learning rate during training to observe its changes. You can access the current learning rate using `optimizer.param_groups[0]['lr']`.

Learning Rate Scheduling in Pytorch

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 from torch.optim import lr_scheduler
5 from torch.utils.data import DataLoader, TensorDataset
6
7 # Dummy data
8 X = torch.randn([100, 10])
9 y = torch.randint(0, 2, (100,))
10 dataset = TensorDataset(X, y)
11 dataloader = DataLoader(dataset, batch_size=16)
12
13 # Simple model
14 class SimpleNet(nn.Module):
15     def __init__(self, input_size, output_size):
16         super(SimpleNet, self).__init__()
17         self.fc = nn.Linear(input_size, output_size)
18
19     def forward(self, x):
20         return torch.sigmoid(self.fc(x))
21
22 model = SimpleNet(input_size=10, output_size=1)
23 criterion = nn.BCELoss()
24 optimizer = optim.Adam(model.parameters(), lr=0.01)
25
```

```
26 # Initialize StepLR scheduler
27 scheduler = lr_scheduler.StepLR(optimizer, step_size=10, gamma=0.1)
28
29 num_epochs = 30
30
31 for epoch in range(num_epochs):
32     for i, (inputs, labels) in enumerate(dataloader):
33         # Forward pass
34         outputs = model(inputs)
35         loss = criterion(outputs, labels.float().unsqueeze(1))
36
37         # Backward and optimize
38         optimizer.zero_grad()
39         loss.backward()
40         optimizer.step()
41
42         if (i + 1) % 10 == 0:
43             print(f'Epoch [{epoch+1}/{num_epochs}], Step [{i+1}/{len(dataloader)}], \
44                   Loss: {loss.item():.4f}, LR: {optimizer.param_groups[0]["lr"]:.6f}')
45
46     # Step the scheduler (for StepLR, MultiStepLR, ExponentialLR, CosineAnnealingLR, CyclicLR, OneCycleLR)
47     scheduler.step()
48
49     # For ReduceLROnPlateau, you would typically step based on a validation metric:
50     # val_loss = evaluate(model, validation_loader)
51     # scheduler.step(val_loss)
52
53 print("Training finished")
```

Data Handling in Pytorch

PyTorch provides powerful and flexible tools for managing and processing your data during training. The core components involved are:

- **torch.utils.data.Dataset:** An abstract class representing a dataset. You need to implement custom dataset classes that define how to access your data and labels.
- **torch.utils.data.DataLoader:** An iterator that provides batches of data from a **Dataset**. It handles shuffling, batching, and parallel data loading.
- **torchvision.transforms (for image data):** A module containing common image transformations that can be used for data augmentation.

1. Batching Strategies in PyTorch:

PyTorch makes batching straightforward using the **DataLoader**.

- **torch.utils.data.DataLoader:** This class takes a **Dataset** object as input and provides an iterable over the data in batches.
- **Key Parameters for Batching:**
 - **batch_size (int, optional):** How many samples per batch to load (default: 1).
 - **drop_last (bool, optional):** If **True**, the last incomplete batch is dropped if its size is less than **batch_size**. Defaults to **False**.

Data Handling in Pytorch

2. Data Augmentation in PyTorch:

For common data types like images, PyTorch provides the `torchvision.transforms` module. You can define a sequence of transformations to apply to your data.

- **`torchvision.transforms`:** This module offers a wide range of image transformations.
- **Common Augmentation Transforms:**
 - **`transforms.ToTensor()`:** Converts a PIL Image or NumPy ndarray into a PyTorch Tensor.
 - **`transforms.Normalize(mean, std)`:** Normalizes a tensor image with mean and standard deviation.
 - **`transforms.RandomHorizontalFlip(p=0.5)`:** Randomly flips the image horizontally with a given probability.
 - **`transforms.RandomVerticalFlip(p=0.5)`:** Randomly flips the image vertically with a given probability.
 - **`transforms.RandomRotation(degrees)`:** Rotates the image by a random angle within the specified degrees.
 - **`transforms.RandomResizedCrop(size, scale=(0.08, 1.0), ratio=(3/4, 4/3))`:** Crops a random sized and aspect ratio patch of the original image and then resizes it to the given size.
 - **`transforms.ColorJitter(brightness=0, contrast=0, saturation=0, hue=0)`:** Randomly changes the brightness, contrast, saturation, and hue of an image.
 - **`transforms.GaussianBlur(kernel_size, sigma=(0.1, 2.0))`:** Applies Gaussian blur to the image.
 - **`transforms.Compose(transforms)`:** Chains multiple transforms together.
- **Applying Augmentations:** You typically define a `transform` object using `transforms.Compose` and then apply it within your custom `Dataset` class's `__getitem__` method or when creating the `DataLoader` (though applying within `Dataset` is more common).
- **Custom Augmentations:** For more specific or complex augmentations, you can create your own custom transformation classes by inheriting from `torch.nn.Module` and implementing the `__call__` method.

Data Handling in Pytorch

3. Data Shuffling in PyTorch:

Shuffling the training data in each epoch is crucial to prevent the model from learning spurious patterns based on the order of the data.

- **`torch.utils.data.DataLoader`:** The `DataLoader` handles shuffling.
- **Key Parameter for Shuffling:**
 - **`shuffle` (bool, optional):** Set to `True` to have the data reshuffled at every epoch (default: `False`).

4. Combining Batching, Augmentation, and Shuffling:

In a typical PyTorch training pipeline, you'll combine these three aspects:

1. **Create a `Dataset`:** This will load your data and apply any necessary transformations (including data augmentation if applicable).
2. **Create a `DataLoader`:** This will take your `Dataset` and handle batching and shuffling.

5. Handling Different Data Types:

- **Text Data:** For text, you might use libraries like `torchtext` or Hugging Face's `Transformers` library, which provide their own `Dataset` and `DataLoader` implementations along with text-specific augmentation techniques.
- **Audio Data:** Libraries like `torchaudio` offer datasets and transformations relevant to audio processing.

In essence, the core principles remain the same: create a `Dataset` to manage your data and apply transformations, and use a `DataLoader` to handle batching and shuffling during training. The specific implementation details and augmentation techniques will vary depending on the data modality.

Data Handling in Pytorch

```
1 import torch
2 from torch.utils.data import Dataset, DataLoader, TensorDataset
3 from torchvision import transforms
4
5 # Dummy data
6 data = torch.randn(100, 3, 32, 32) # Example image data (100 images, 3 channels, 32x32)
7 labels = torch.randint(0, 10, (100,)) # 10 classes
8
9 # Define image transformations (including augmentation)
10 transform = transforms.Compose([
11     transforms.RandomHorizontalFlip(),
12     transforms.RandomRotation(degrees=10),
13     transforms.ToTensor(),
14     transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
15 ])
16
17 # Create a custom Dataset (if needed) or use a built-in one
18 class CustomDataset(Dataset):
19     def __init__(self, data, labels, transform=None):
20         self.data = data
21         self.labels = labels
22         self.transform = transform
23
24     def __len__(self):
25         return len(self.data)
26
27     def __getitem__(self, idx):
28         sample = self.data[idx]
29         label = self.labels[idx]
30         if self.transform:
31             sample = self.transform(sample) # Assuming your data loading handles PIL Images or similar
32         return sample, label
33
34 dataset = CustomDataset(data, labels, transform=transform)
35
36 # Create a DataLoader with batch size and shuffling
37 batch_size = 16
38 shuffle = True
39 dataloader = DataLoader(dataset, batch_size=batch_size, shuffle=shuffle)
```

Regularization Techniques

Dropout is a regularization technique where randomly selected neurons are "dropped out" (set to zero) during the training process. This means their contribution to the activation of downstream neurons is temporarily removed. The probability of a neuron being dropped out is controlled by a hyperparameter, typically denoted as 'p'.

→ Implementation in PyTorch:

- **torch.nn.Dropout layer:** Show how to add dropout layers to a model.
- **p parameter:** Explain the dropout probability.
- **torch.nn.Dropout(p=0.5, inplace=False):**
- **Training vs. Evaluation:** Emphasize that dropout is active during training but turned off during evaluation (`model.eval()`).

Weight decay is a technique that adds a penalty to the loss function proportional to the square of the magnitude of the model's weights.

→ Implementation in PyTorch: **weight_decay parameter in optimizer** (`torch.optim.Adam` or `torch.optim.SGD`.)

Early stopping is a regularization technique that involves monitoring the model's performance on a validation set during training. Training is stopped prematurely when the performance on the validation set starts to degrade (e.g., the validation loss starts increasing or the validation accuracy starts decreasing), even if the training loss is still decreasing.

→ Implementation in PyTorch: Early stopping is typically implemented manually within your training loop.

1. Keep track of the validation loss (or another relevant metric).
2. Define a "patience" value: This is the number of epochs to wait after the validation loss has stopped improving before stopping training.
3. Keep track of the best validation loss seen so far and the corresponding model state.
4. In each epoch:
 - Train on the training data.
 - Evaluate on the validation data and calculate the validation loss.
 - If the current `val_loss` is better than the best `val_loss` seen so far, update the best loss and save the current model state.
 - If the `val_loss` has not improved for 'patience' epochs, stop training.

Regularization Techniques

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 from torch.utils.data import DataLoader, TensorDataset
5
6 # Assume you have your model,
7 # optimizer, loss function, training_loader, and validation_loader
8
9 # Example data and loaders (replace with your actual data)
10 X_train = torch.randn(80, 10)
11 y_train = torch.randint(0, 2, (80,))
12 train_dataset = TensorDataset(X_train, y_train)
13 train_loader = DataLoader(train_dataset, batch_size=16)
14
15 X_val = torch.randn(20, 10)
16 y_val = torch.randint(0, 2, (20,))
17 val_dataset = TensorDataset(X_val, y_val)
18 val_loader = DataLoader(val_dataset, batch_size=16)
19
20 model = nn.Linear(10, 1)
21 optimizer = optim.Adam(model.parameters(), lr=0.01)
22 criterion = nn.BCEWithLogitsLoss()
23
24 num_epochs = 100
25 patience = 10
26 best_val_loss = float('inf')
27 epochs_no_improve = 0
28 best_model_state = None
```

```
30 for epoch in range(num_epochs):
31     model.train()
32     for inputs, labels in train_loader:
33         optimizer.zero_grad()
34         outputs = model(inputs)
35         loss = criterion(outputs, labels.float().unsqueeze(1))
36         loss.backward()
37         optimizer.step()
38
39     model.eval()
40     val_loss = 0.0
41     with torch.no_grad():
42         for inputs, labels in val_loader:
43             outputs = model(inputs)
44             loss = criterion(outputs, labels.float().unsqueeze(1))
45             val_loss += loss.item() * inputs.size(0)
46     val_loss /= len(val_loader.dataset)
47     print(f'Epoch [{epoch+1}/{num_epochs}], Validation Loss: {val_loss:.4f}')
48
49     if val_loss < best_val_loss:
50         best_val_loss = val_loss
51         epochs_no_improve = 0
52         best_model_state = model.state_dict()
53     else:
54         epochs_no_improve += 1
55         if epochs_no_improve == patience:
56             print(f'Early stopping triggered after {epoch+1} epochs!')
57             model.load_state_dict(best_model_state) # Load the best model
58             break
59
```

Gradient Clipping

Gradient Clipping as a Solution in PyTorch. Gradient clipping sets a threshold on the magnitude of the gradients. If the gradients exceed this threshold, they are scaled down to prevent overly large weight updates. PyTorch provides convenient utilities in the `torch.nn.utils` module to implement gradient clipping.

There are **two main ways to implement gradient clipping in PyTorch**:

1. **Clipping by Value (`torch.nn.utils.clip_grad_value_`)**: This method directly clips the individual values of the gradients to a specified range.
2. **Clipping by Norm (`torch.nn.utils.clip_grad_norm_`)**: This is the more common and often recommended approach. It clips the **L2 norm** (or another specified norm) of the gradients of all parameters together. If the total norm exceeds a threshold, all gradients are scaled down proportionally.

Gradient clipping is particularly useful in the following scenarios:

- **Recurrent Neural Networks (RNNs)**: RNNs, especially those with many time steps or complex architectures like LSTMs and GRUs, are prone to exploding gradients.
- **Deep Neural Networks**: Very deep feedforward networks can sometimes experience this issue as well.
- **Training with High Learning Rates**: If you are using a relatively high learning rate, gradient clipping can help maintain stability.
- **Observing Unstable Training**: If you notice your training loss fluctuating wildly or increasing, it might be a sign of exploding gradients, and gradient clipping could help.

Implementation Steps in the Training Loop:

1. After calculating the gradients using `loss.backward()`.
2. Before the optimizer's `step()` function updates the model's parameters.

Choosing the Clipping Threshold:

The optimal clipping threshold (`clip_value` for value clipping or `max_norm` for norm clipping) often needs to be determined through experimentation. You can try different values and monitor the training process (e.g., loss curves, gradient magnitudes) to find a suitable threshold that stabilizes training without hindering learning. Common values for `max_norm` often range between 0.1 and 10.

Gradient Clipping

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 from torch.utils.data import DataLoader, TensorDataset
5
6 # Example data and model
7 X = torch.randn(100, 10)
8 y = torch.randint(0, 2, (100,))
9 dataset = TensorDataset(X, y)
10 dataloader = DataLoader(dataset, batch_size=16)
11
12 model = nn.Linear(10, 1)
13 optimizer = optim.Adam(model.parameters(), lr=0.01)
14 criterion = nn.BCEWithLogitsLoss()
15
16 num_epochs = 30
17 clip_norm = 1.0
18
19 for epoch in range(num_epochs):
20     for inputs, labels in dataloader:
21         optimizer.zero_grad()
22         outputs = model(inputs)
23         loss = criterion(outputs, labels.float().unsqueeze(1))
24         loss.backward()
25
26         # Clip gradients by norm
27         torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=clip_norm)
28
29         optimizer.step()
30
31         # Optional: Monitor gradient norms
32         # total_norm = torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=clip_norm, norm_type=2)
33         # print(f"Gradient Norm: {total_norm.item():.4f}")
34
35     print(f'Epoch [{epoch+1}/{num_epochs}] finished.')
36
37 print(["Training finished"])
```

Debugging and Logging: Tools and techniques for understanding the training process.

Practice 3 - Get started with Hugging Face

Exercise1: Sentiment Analysis with Hugging Face

1. Install the Hugging Face `transformers` library.
2. Use a pre-trained sentiment analysis model from the Hugging Face Hub.
3. Tokenize a sample sentence.
4. Perform sentiment analysis on the sentence.

References:

<https://huggingface.co/docs/transformers/en/training#fine-tune-a-pretrained-model>

<https://huggingface.co/blog/sentiment-analysis-python>

<https://www.kaggle.com/code/gauravduttakiit/sentiment-analysis-using-hugging-face>

<https://www.kaggle.com/code/neerajmohan/fine-tuning-bert-for-text-classification>

Exercise2: Finetuning a Pretrained Model for Binary Text Classification

In this exercise, you will:

1. Install the necessary Hugging Face libraries (`transformers`, `datasets`, `evaluate`).
2. Load a simple dataset for binary text classification.
3. Load a pretrained model and its tokenizer.
4. Preprocess the dataset to be suitable for the model.
5. Define training arguments.
6. Create a `Trainer` object and finetune the model.
7. Evaluate the finetuned model.

Q & A